

Exercise Sheet

Task 1 – Dependencies

Why is read after read not a dependency?

Say we have two instructions I1 and I2 that both read register t0 immediately after each other. Both I1 and I2 depend on the value in register t0, that is, unless other dependencies come up, not changed by either of these instructions and can therefore be read without problems.

Task 2 – Pipelining

a) Determine the dependencies of the following instructions:

I1: ADD t0, t0, t1
 I2: SUB t1, t0, t2
 I3: BNE t3, t1, jump
 I4: ADD t4, t2, t3
 jump:
 I5: LW t4, 24(t0)
 I6: SLLI t3, t3, 4
 I7: ADD t5, t4, t5
 I8: XOR t0, t2, t3
 I9: SW t1, 12(t3)

	I1	I2	I3	I4	I5	I6	I7	I8	I9
I1		A(t1), T(t0)			T(t0)			O(t0), A(t0)	
I2			T(t1)					A(t0)	T(t1)
I3						A(t3)			
I4					O(t4)	A(t3)	T(t4)		
I5							T(t4)	A(t0)	
I6								T(t3)	T(t3)
I7									
I8									
I9									

b) How would the program be executed in a 5-stage pipeline without forwarding or branch-prediction if the branch was not taken?

IF	ID	EX	MA	WB
I1				
NOP	I1			
NOP	NOP	I1		
NOP	NOP	NOP	I1	
I2	NOP	NOP	NOP	I1
NOP	I2	NOP	NOP	NOP
NOP	NOP	I2	NOP	NOP
NOP	NOP	NOP	I2	NOP
I3	NOP	NOP	NOP	I2
I4	I3	NOP	NOP	NOP
I5	I4	I3	NOP	NOP
I6	I5	I4	I3	NOP
NOP	I6	I5	I4	I3
NOP	NOP	I6	I5	I4
I7	NOP	NOP	I6	I5
I8	I7	NOP	NOP	I6
I9	I8	I7	NOP	NOP
	I9	I8	I7	NOP
		I9	I8	I7
			I9	I8
				I9

c) How many NOPs less are needed if forwarding is possible?

When forwarding is possible, 3 NOPs less are needed.

Task 3 – Scoreboarding

Our processor consists of one functional unit for addition and one for multiplication. There is no forwarding, therefore an instruction can only access the value computed by another instruction after it has left the “write back”-stage. In what order will the following instructions be executed?

I1: ADD t0, t0, t1
I2: MUL t1, t0, t1
I3: MUL t2, t2, t3
I4: SUB t3, t0, t3
I5: MUL t4, t0, t1
I6: ADD t5, t0, t3
I7: ADD t5, t0, t2

For each instruction that is currently executed, indicate which registers and functional units are currently reserved or in use, and indicate if instructions are executed at the same time.

Instructions in Issue Stage	Instructions in Execution	FU(+)	FU(*)	t0	t1	t2	t3	t4	t5
I1									
I2	I1	I1		I1					
I2, I3				I1					
I2, I4	I3		I3	I1		I3			
I5	I2, I4	I4	I2		I2	I3	I4		
I5, I6					I2	I3	I4		
I5, I6, I7					I2		I4		
I7	I5, I6	I5	I6					I5	I6
I7								I5	I6
I7								I5	I6
	I7	I7							I7
									I7
									I7